

CROSS-SITE REQUEST FORGERY (CSRF)

INTRODUCTION

In today's interconnected world, web applications handle sensitive user actions such as online banking transactions, profile updates, and order processing. These applications often rely on cookies and sessions to identify authenticated users. While this improves user convenience, it also opens the door to specific security threats like Cross-Site Request Forgery (CSRF). CSRF is a web vulnerability that manipulates the trust a website has in a user's browser, allowing attackers to trick authenticated users into performing unintended actions. Understanding CSRF, how it works, its testing methodologies, and mitigation techniques is critical for maintaining the integrity, confidentiality, and security of web applications.

1. What is CSRF?

Cross-Site Request Forgery (CSRF) is a web application vulnerability that forces an end user to execute unwanted actions on a web application where they are authenticated. It takes advantage of the implicit trust that a website places in a user's browser and session.

In simpler terms, an attacker tricks a logged-in user into performing an action they did not intend, such as changing account details, transferring funds, or deleting data, without their explicit consent.

CSRF is sometimes called a "one-click attack" or "session riding".

2. How does a CSRF attack work?

A CSRF attack typically follows this sequence:

1. Victim logs in to a legitimate website (e.g., online banking). The site sets an authentication cookie in the browser.
2. Victim visits a malicious website controlled by the attacker or clicks on an attacker-sent link (email, chat, etc.).
3. The malicious site contains hidden requests (forms, scripts, images) that point to the target application.

4. Browser automatically sends cookies/session data with the request because the victim is still logged in.
5. The server processes the request as if it was sent by the victim intentionally.

Example:

```

```

If the victim is logged in to *bank.com*, their browser sends the authenticated request, transferring money to the attacker.

3. How do you test for CSRF vulnerability?

a) Manual Testing

- Identify sensitive requests (e.g., password changes, fund transfers).
- Remove or modify the CSRF token in the request and resend it to check if it still executes.
- Copy the request and send it from a different origin to see if it's accepted.

b) Automated Testing Tools

- Burp Suite – Generates CSRF proof-of-concept HTML automatically.
- OWASP ZAP – Detects potential CSRF risks in scanning.
- Postman – Used for crafting custom test requests.

4. Impact and Mitigations

Impact of CSRF:

- Financial fraud (unauthorized transfers, purchases)
- Account takeover (email/password changes)
- Mass actions (deleting all data, modifying privileges)
- Reputation damage for the affected service

General Mitigation Strategies:

- Enforce CSRF tokens for all state-changing requests.
- Use SameSite cookies to prevent cross-origin request forgery.
- Require user re-authentication for critical actions.
- Check the *Origin* and *Referer* headers to ensure requests come from trusted sources.

5. How to Mitigate CSRF Vulnerabilities

1. CSRF Tokens – Use strong, random, per-session tokens embedded in forms and verified server-side.

2. Same Site Cookies – Set authentication cookies to *SameSite=Lax* or *SameSite=Strict*.

3. Double Submit Cookie Pattern – Send a token in both a cookie and a request parameter, verifying they match.

4. Re-authentication & MFA – Request the user's password or OTP before critical actions.

5. Framework Security Features – Use built-in CSRF protection from frameworks like:

- *Django*: `@csrf_protect`
- *Laravel*: CSRF middleware
- *Spring Security*: CSRF filter enabled by default

CONCLUSION

CSRF is a silent but powerful attack that abuses the trust between a user's browser and a web application. While it can be devastating, awareness, secure coding practices, and modern browser security features can almost completely eliminate the threat. Implementing strong CSRF protections is a necessity for every web application that handles sensitive user actions.